

**Автономная некоммерческая организация высшего образования  
«Университет Иннополис»**

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА  
(БАКАЛАВРСКАЯ РАБОТА)  
по направлению подготовки  
09.03.01 «Информатика и вычислительная техника»**

**FINAL QUALIFICATION WORK  
(BACHELOR'S THESIS)  
Academic Program  
09.03.01 «Computer Science»**

**Направленность (профиль) образовательной программы  
«Информатика и вычислительная техника»**

**Field of Study:  
«Computer Science»**

**Тема**

**Разработка системы мониторинга производительности и  
обнаружения регрессий в Android-приложениях**

**Topic**

**Development of a Performance Monitoring and Regression  
Detection System for Android Applications**

Работу выполнил /  
*Prepared by*

**Миннемуллин Наиль  
Фидаилевич /  
Nail Minnemullin**

подпись / signature

Руководитель  
выпускной  
квалификационной  
работы / *Final  
Qualification Work  
Supervisor*

**Маццара Мануэль /  
Manuel Mazzara**

подпись / signature

Иннополис, Innopolis, 2026

# Contents

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                   | <b>7</b>  |
| 1.1      | Background and motivation . . . . .                   | 7         |
| 1.2      | Research Gap . . . . .                                | 8         |
| 1.3      | Research Questions and Aim . . . . .                  | 8         |
| 1.4      | Novelty and Contribution . . . . .                    | 9         |
| 1.5      | Structure . . . . .                                   | 10        |
| <b>2</b> | <b>Literature Review</b>                              | <b>11</b> |
| 2.1      | Performance monitoring . . . . .                      | 12        |
| 2.2      | Approaches for Performance Data Collection . . . . .  | 12        |
| 2.2.1    | Pre-release Profiling . . . . .                       | 12        |
| 2.2.2    | Post-release Monitoring . . . . .                     | 14        |
| 2.3      | Performance Metrics . . . . .                         | 15        |
| 2.4      | Performance Regression Detection . . . . .            | 17        |
| 2.4.1    | Approaches to Detecting Performance Regressions . . . | 17        |
| 2.4.2    | Mobile-Specific Considerations and Research Gap . . . | 19        |
| 2.5      | Summary . . . . .                                     | 20        |
| <b>3</b> | <b>Methodology</b>                                    | <b>21</b> |

---

|          |                                                   |           |
|----------|---------------------------------------------------|-----------|
| 3.1      | System requirements and constraints . . . . .     | 21        |
| 3.2      | High-Level System Architecture . . . . .          | 22        |
| 3.3      | Mobile SDK Design . . . . .                       | 23        |
| 3.3.1    | Metrics Selection and Collection . . . . .        | 23        |
| 3.3.2    | UI performance metrics . . . . .                  | 23        |
| 3.3.3    | CPU usage . . . . .                               | 25        |
| 3.3.4    | Memory usage . . . . .                            | 26        |
| 3.3.5    | Data Aggregation and Transmission . . . . .       | 26        |
| 3.4      | Backend Design . . . . .                          | 27        |
| 3.4.1    | Database Design . . . . .                         | 27        |
| 3.4.2    | Data Partitioning and Cohorts . . . . .           | 30        |
| 3.4.3    | Performance Regression Detection . . . . .        | 30        |
| 3.5      | Summary . . . . .                                 | 32        |
| <b>4</b> | <b>Implementation</b>                             | <b>33</b> |
| 4.1      | Mobile SDK Implementation . . . . .               | 33        |
| 4.1.1    | Initialization and Lifecycle Management . . . . . | 34        |
| 4.1.2    | Performance Metrics Collectors . . . . .          | 35        |
| 4.1.3    | Data Buffering and Network Transmission . . . . . | 38        |
| 4.2      | Backend Implementation . . . . .                  | 40        |
| 4.2.1    | User and Project Management . . . . .             | 40        |
| 4.2.2    | Data Ingestion . . . . .                          | 41        |
| 4.3      | Performance Regression Detection . . . . .        | 43        |
| 4.4      | Frontend Implementation . . . . .                 | 45        |
| 4.4.1    | User Authorization . . . . .                      | 46        |
| 4.4.2    | Project Management . . . . .                      | 46        |

|                                                   |           |
|---------------------------------------------------|-----------|
| <b>CONTENTS</b>                                   | <b>4</b>  |
| 4.4.3 Metrics Explorer . . . . .                  | 47        |
| 4.4.4 Regressions . . . . .                       | 48        |
| 4.5 Summary . . . . .                             | 50        |
| <b>5 Evaluation and Discussion</b>                | <b>51</b> |
| 5.1 Validation Setup . . . . .                    | 51        |
| 5.1.1 Test Devices . . . . .                      | 52        |
| 5.1.2 Test Application . . . . .                  | 52        |
| 5.2 SDK Overhead Evaluation . . . . .             | 53        |
| 5.3 Metric Validity . . . . .                     | 55        |
| 5.4 Regression Detection Evaluation . . . . .     | 55        |
| 5.4.1 Experiment Setup . . . . .                  | 56        |
| 5.4.2 Regression Injection . . . . .              | 56        |
| 5.4.3 Detection of Injected Regressions . . . . . | 57        |
| 5.4.4 Threshold Sensitivity . . . . .             | 59        |
| 5.5 Discussion of Results . . . . .               | 61        |
| 5.6 Limitations . . . . .                         | 62        |
| <b>6 Conclusion</b>                               | <b>63</b> |
| 6.1 Future Work . . . . .                         | 64        |
| <b>Bibliography cited</b>                         | <b>65</b> |

# List of Tables

|     |                                                                                                                    |    |
|-----|--------------------------------------------------------------------------------------------------------------------|----|
| 5.1 | Test Devices Used in the Evaluation . . . . .                                                                      | 52 |
| 5.2 | SDK Overhead Measured on the Test Devices . . . . .                                                                | 53 |
| 5.3 | Cold-Start Time Overhead Measured on the Test Devices (mean<br>of ten launches) . . . . .                          | 53 |
| 5.4 | P95 Shift and Detection Outcome for the Injected Regressions<br>(Release-Pair Comparison, 15% Threshold) . . . . . | 57 |
| 5.5 | Detector Quality vs. Regression Threshold . . . . .                                                                | 59 |

# List of Figures

|     |                                                                                                                                                                                            |    |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.1 | High-Level System Architecture . . . . .                                                                                                                                                   | 22 |
| 3.2 | Sequence Diagram of SDK Data Collection and Transmission .                                                                                                                                 | 27 |
| 3.3 | Relational data model used by the backend . . . . .                                                                                                                                        | 29 |
| 4.1 | Login page of the dashboard . . . . .                                                                                                                                                      | 46 |
| 4.2 | Project list page . . . . .                                                                                                                                                                | 47 |
| 4.3 | Metrics Explorer: filter selection . . . . .                                                                                                                                               | 47 |
| 4.4 | Metrics Explorer: frame time chart with a detected regression .                                                                                                                            | 48 |
| 4.5 | Regressions page of the dashboard . . . . .                                                                                                                                                | 49 |
| 4.6 | Regression alert delivered to a messaging app . . . . .                                                                                                                                    | 49 |
| 5.1 | SDK overhead charts for all four test devices. Each chart shows mean CPU usage, PSS memory, and cold-start time without and with the SDK enabled. . . . .                                  | 54 |
| 5.2 | Detection rate by metric, injection intensity, and device cohort. Each bar shows the fraction of runs in that cell that the detector flagged as a regression at the 15% threshold. . . . . | 58 |

---

|     |                                                                                                                                                                                                                         |    |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 5.3 | Detector quality as a function of the relative-shift threshold. TPR and FPR are computed on the 30 injected runs and 15 control runs respectively. The vertical dashed line marks the default threshold of 15%. . . . . | 60 |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|

## **Abstract**

Mobile applications are updated frequently, and a new release can quietly degrade performance on real user devices, an effect known as a performance regression. Detecting such regressions is difficult because Android devices vary widely in hardware and operating system version, and because existing tools address only part of the problem. Pre-release profilers run under controlled conditions and add too much overhead for continuous use, while production monitoring tools either provide no automated alerting or rely on fixed absolute thresholds that must be set by hand for each metric and rarely fit every device. Release-to-release regression detection, in turn, has been studied mainly for homogeneous server software rather than for fragmented Android fleets. This thesis develops a system that combines continuous low-overhead post-release monitoring on Android with automated server-side regression detection. An embeddable Kotlin SDK collects user-perceived and resource metrics on real devices and uploads them to a Ktor backend in batches, where they are stored in ClickHouse and PostgreSQL and grouped by device performance cohort. A detection pipeline compares each release against the previous one using the relative shift of the 95th percentile, grouped by metric, screen, and cohort, and reports a regression when the shift exceeds a single relative threshold. This removes the need to calibrate absolute per-metric limits and attributes every alert to a specific release. The system was evaluated for runtime overhead and for detection accuracy. Enabling the SDK added at most 5.2 percentage points of CPU usage, 5.4 MB of memory, and 75 ms of cold-start time across the tested devices. On synthetic regressions spanning three types, three intensities, and three device cohorts, the pipeline detected 27 of

30 cases (90 percent) at the default 15 percent threshold, with one false alert in 15 control runs. The results show that continuous monitoring and automated, cohort-aware regression detection can be delivered together on real Android devices at a cost that users are unlikely to notice.

# Chapter 1

## Introduction

### 1.1 Background and motivation

Mobile applications are now an essential part of everyday life, and users expect them to be fast, responsive, and stable. Poor performance (e.g. slow startup, high CPU usage, or freezing UI) can quickly degrade the user experience and result in negative reviews [1], [2]. Because of this, performance monitoring has become a crucial part of modern software development. Applications are also updated frequently, and any new release can unintentionally degrade performance on users' devices. This kind of release-to-release degradation, known as a performance regression, may stay invisible during development and appear only after the update reaches real users.

A main challenge in measuring Android application performance is fragmentation: the large variety of devices, operating system versions, and hardware configurations. The same application may perform differently across devices, so the most reliable way to understand its real-world behaviour is to observe it on real user devices under real usage conditions [3], [4]. Monitoring performance in

this way raises two further difficulties. First, because it runs on real user devices, the monitoring must add little runtime overhead, so that measuring performance does not itself harm the user experience it is meant to protect. Second, such tools often lack automated alerting: they either provide no alerting at all or rely on thresholds that must be set manually for each metric and application and can easily lead to inaccurate detection. As a result, developers need a way to monitor how an application behaves on real devices continuously and with low overhead, and to detect automatically when a new release degrades performance.

## **1.2 Research Gap**

Existing Android performance tools do not combine continuous monitoring on real devices with automated regression detection. Tools that do provide alerting, such as Firebase Performance Monitoring, rely on fixed absolute thresholds that a developer must set manually for each metric, which can lead to a large number of false positives or false negatives [2]. At the same time, studies on regression detection are developed independently and are not connected with continuous monitoring of Android applications. As a result, there is no existing solution that combines continuous post-release performance monitoring with server-side regression detection. This gap is what the current work aims to address.

## **1.3 Research Questions and Aim**

The aim of this thesis is to develop a system that continuously collects performance metrics from running Android applications and detects performance

regressions introduced by new releases. It gathers key metrics directly from users' devices, sends the data to a backend, and notifies developers when a regression is detected.

The study addresses the following research questions:

1. How can Android application performance be monitored continuously with low runtime overhead?
2. How can performance regressions introduced by new application releases be detected automatically?
3. How accurate is the proposed system in detecting performance regressions?

## **1.4 Novelty and Contribution**

The novelty of this work is in combining post-release Android performance monitoring with automated performance regression detection in a single tool. The proposed solution is a low-overhead, continuous monitoring system designed for Android's fragmented device ecosystem. Instead of relying on static thresholds tuned per metric, the system detects regressions by comparing performance metrics between consecutive application releases, stratifying the comparison by metric, screen, and device performance cohort. As a result, developers receive notifications that are attributed to a specific release and that account for hardware heterogeneity, allowing them to address performance issues before they affect a significant number of users.

## 1.5 Structure

The thesis is organised as follows.

Chapter 2 reviews related work on mobile performance monitoring and regression detection. It compares pre-release profiling with post-release monitoring, looks at which performance metrics earlier studies relied on, and sets out the research gap behind this work.

Chapter 3 turns to the design of the proposed system. It covers the requirements, the overall architecture, the mobile SDK and the backend, and the regression detection methods that were weighed before one was chosen.

Chapter 4 describes how the system was built, from the mobile SDK and the backend to the frontend dashboard and the regression detection pipeline.

Chapter 5 evaluates the proposed system. It describes the validation setup, measures the overhead introduced by the SDK and evaluates the regression detection pipeline on synthetic regressions injected into a demo application, and discusses the results in relation to the research questions, together with their limitations.

Finally, Chapter 6 concludes the thesis by restating the problem and what was done, explaining the usefulness of the work, and outlining directions for future work.

## Chapter 2

# Literature Review

This chapter reviews existing research on mobile application profiling and monitoring. It looks at how performance metrics are collected and at which metrics matter most for mobile development. It then examines the main approaches to performance regression detection, which prepares the choice of method made later in this work.

The review is organised into three main parts. The first part compares the two main approaches to collecting performance metrics in mobile applications. The second part summarises the metrics used in previous research and explains how they contribute to the assessment of application performance and user experience. The third part reviews existing approaches to performance regression detection, from manual threshold-based alerting to release-to-release comparison in large-scale production systems. Together, these parts show the current state of research and provide a foundation for identifying the gap that the proposed system aims to address.

## 2.1 Performance monitoring

Performance strongly shapes how users experience a mobile application and whether they keep using it. Slow response times, high energy use, or a laggy interface frustrate users and often end in poor reviews and uninstalls [5]. Catching such problems early keeps an application usable and its users satisfied, as Tang *et al.* [1] demonstrate. Two strategies are commonly used to understand application performance: profiling and monitoring. Profiling runs during development or testing and yields detailed, high-resolution data. Monitoring, in contrast, takes place after the application is released and runs on real user devices. It must introduce as little overhead as possible to prevent any interference with normal app usage.

The Android ecosystem adds complexity to performance evaluation due to fragmentation - the wide variety of hardware, operating system versions, and user behaviours [4]. This diversity makes it difficult to reproduce rare or device-specific issues found in the wild. The same app may behave differently on various devices depending on their environment, OS version, and hardware specifications [3]. This problem motivates research into tools and methods for collecting and analysing mobile performance data effectively.

## 2.2 Approaches for Performance Data Collection

### 2.2.1 Pre-release Profiling

Several tools have been developed to support performance profiling during development. One example is ProfileDroid by Wei *et al.* [6], which uses a

multi-layer approach to profiling. It gathers data from four layers: application metadata, user-interface interaction traces, operating-system metrics such as CPU and memory usage, and network activity. Looking at these layers together lets developers see how the parts of an application interact, for instance how a single UI event triggers a CPU spike or a network request. The main strength of ProfileDroid is its ability to correlate performance across layers, giving a complete system-level view. However, its high overhead and dependence on a controlled environment limit its practical use on real devices.

Another relevant project is AndroStep [7], which focuses on analysing Android storage performance during I/O operations. The authors found that existing Android benchmarking tools did not simulate realistic app behaviour, especially missing operations like `fsync()` followed by 4 KB random writes - a common and essential operation used by SQLite, the embedded relational database built into Android for local data storage [7]. The tool is useful for benchmarking Android storage, but it measures nothing beyond storage and has to be run by hand, which limits it to a test environment rather than a live one.

PowDroid, introduced by Bouaffar *et al.* in 2021 [8], was designed to measure the energy consumption of applications. However, like most pre-release tools, it works only under controlled conditions, which makes it difficult to test applications on a large range of real devices.

In the field of UI performance, a new state-of-the-art work, GuiWatcher, was proposed by Liu *et al.* [9]. They presented an advanced way to detect UI lags and freezes. GuiWatcher uses computer vision techniques on screen recordings and is able to detect three types of problematic frames: "junky" frames (dropped or delayed ones), long loading frames (lengthy transition), and frozen frames. This method reflects what users actually see rather than relying only on timing

logs. However, it is computationally expensive and suitable only for use during development or continuous integration testing. It also raises privacy concerns, because it records the device screen.

ADEL, by Zhang *et al.* [10], targets network performance and energy leaks. It extends the platform to tag downloaded data and follow how that data moves through an application, which reveals unnecessary network activity that wastes energy. The tool is not built for continuous monitoring, however, because it modifies the Android system and adds high overhead, so it cannot run on real user devices.

### 2.2.2 Post-release Monitoring

The main idea of post-release profiling is to collect metrics from apps as they run on real user devices. The main advantage of this approach is that it captures realistic usage patterns and real-world diversity that are hard to reproduce in a controlled test environment. The main challenge is to minimise overhead to avoid affecting user experience.

One of the first studies in this area was AppInsight [11], which introduced a method for instrumenting applications without modifying their source code. The framework automatically injects instrumentation into the compiled bytecode and collects runtime traces. By collecting traces from 30 users over several months, the authors were able to identify performance bottlenecks that were not visible during development. However, this approach is limited because developers cannot easily add custom metrics.

Another example is Hubble [12], which is currently integrated into all Huawei devices. The main feature of Hubble is that it monitors the performance in an

efficient in-memory buffer, where older data is constantly overwritten [12]. The relevant metrics are only persisted when a performance issue is detected, providing engineers with detailed runtime traces from just before it occurred. This approach combines high efficiency with low storage cost.

PerfProbe [13] works across layers to find the root cause of a performance problem rather than only its symptom. It traces both the application and the system on real devices at low overhead, then uses statistical analysis to flag functions that behave abnormally and tie them to resource factors such as CPU, memory, or I/O. A developer thus learns not only where a slowdown happens but why. Across 22 Android apps, PerfProbe diagnosed real issues and reduced user-perceived latency in six of them by 32% to 86% [13].

Panappticon [14] is a lightweight tracing system that records performance across the application, system, and kernel layers. It traces critical execution paths automatically by linking each user input to the display update it produces. Unlike AppInsight, it can also catch problems that arise from interactions between several apps or system components. Over a one-month study its overhead stayed around 6%, low enough for continuous use in the field.

## 2.3 Performance Metrics

Performance is not captured by a single number but by a set of metrics, which fall into two kinds: those that describe how an application uses device resources, and those that describe what the user actually perceives. Resource metrics explain where a performance problem originates, while user-perceived metrics show how strongly it affects the user. The two are linked, because resource pressure usually surfaces as a slower or less responsive interface.

On the resource side, the most common metrics are CPU usage, memory usage, storage I/O, and energy consumption. High CPU utilisation overloads threads and reduces responsiveness, and CPU contention has been shown to be a frequent cause of poor user experience on Android [15]. Memory usage matters for stability: an application that consumes too much RAM can trigger the system's low-memory killer or frequent garbage-collection pauses, so developers profile heap usage to catch allocation bottlenecks and memory leaks [16]. Storage I/O also affects latency, mostly on devices with slower flash memory, where a slow read can make a screen take noticeably longer to open [7]. Energy use is just as visible. Surveys put battery life at the top of users' performance concerns, and people avoid applications they consider too power-hungry [17], so finding energy leaks is an important part of mobile optimisation.

On the user-perceived side, several more specific metrics describe responsiveness. Frame rendering time is the most direct of these. A frame has to be drawn within the display's refresh budget, roughly 16.7 ms on a 60 Hz screen, and a frame that misses this budget shows up as stutter or a dropped frame [18]. Frames per second (FPS) carries the same information in a different form and follows directly from frame time. Startup time, the wait before an application becomes usable after launch, largely sets the user's first impression [19]. Interaction latency is the delay between a user action and the first visible response. Even when CPU and memory look normal, a long delay here noticeably reduces satisfaction, and guidance such as Google's RAIL model treats a response slower than about one second as the point where the user's attention drifts [13]. The most severe responsiveness failure is an Application Not Responding (ANR) event, raised when the main thread stays blocked for several seconds, which often makes the user abandon the application [20].

In practice no single metric is sufficient, and tools combine several to build a fuller picture. ProfileDroid and Panappticon correlate UI and system activity, while PerfProbe links CPU and I/O data with execution paths. The choice of metrics depends on the goal: pre-release profiling collects many low-level metrics, whereas post-release monitoring favours a smaller set of user-perceived metrics, such as frame time, startup time, and interaction latency, that are cheap to measure on real devices and reflect more directly what the user actually experiences.

## **2.4 Performance Regression Detection**

A performance regression is a situation where software still works correctly but performs worse than a previous baseline, for example slower response times or higher resource usage. In mobile applications this usually appears after an update, as increased frame rendering time, slower startup, higher CPU usage, or greater memory consumption. Regression detection aims to identify a sustained degradation in performance across a population of users between two releases, rather than isolated, short-lived spikes.

### **2.4.1 Approaches to Detecting Performance Regressions**

The simplest and most widely used approach in industry is threshold-based alerting. Tools such as Firebase Performance Monitoring [2] and Google Play Vitals [21] let developers set fixed limits on individual metrics, for example a maximum startup time, and raise an alert when a limit is exceeded. This approach is easy to understand, but the thresholds must be set manually for each metric and application, and a single fixed value rarely fits every device and usage pattern,

which leads to false alerts or missed regressions. Such tools also react to absolute values rather than to changes between versions, so a slowdown introduced by a new release goes unnoticed as long as it stays within the configured limit.

A more adaptive approach compares current performance against a historical baseline instead of a fixed limit. Nguyen *et al.* [22] build control limits from a baseline run and flag a new run when it deviates too far from them, which removes the need to choose absolute thresholds by hand. Their method, however, was designed for controlled load-test runs of server software rather than for data collected continuously from real user devices. On mobile, Carat [23] applies a similar idea at the population level: it collects energy data from many users, builds a baseline of normal behaviour for each application, and reports devices and apps that deviate from it. Carat compares devices rather than consecutive releases, so it can locate an abnormal app but does not attribute a regression to a particular version. Industry observability platforms such as Datadog go further and compare live metrics against an automatically learned historical or seasonal baseline, flagging statistically significant deviations [24]. These tools react to anomalies as they unfold over time rather than to the difference between two releases, and they target server and infrastructure metrics rather than mobile applications on user devices.

The approach closest to the needs of this work compares each new release directly against the previous one, so that a detected regression is attributed to a specific version. This is the principle behind canary analysis, which is widely used in industry to gate deployments by comparing a new version against a stable baseline before full rollout [25]. At large scale, Meta's FBDetect [26] monitors hundreds of thousands of time series in production, compares versions to catch even small regressions, and filters out false alerts caused by transient issues such

as load spikes or rolling updates. These systems show that release-to-release comparison is an effective and interpretable way to detect regressions, because each alert is tied to a concrete change. Both systems run inside the data centre, where the software executes on controlled and largely homogeneous servers. They therefore do not deal with device fragmentation and have not been applied to performance data collected from heterogeneous Android devices after release.

### **2.4.2 Mobile-Specific Considerations and Research Gap**

Applying these ideas to Android raises additional constraints. Devices differ widely in hardware and operating system version, so the same release can perform differently across devices, and comparing them directly can make normal hardware differences look like regressions. Devices are also resource-constrained, and collecting detailed traces from real users raises privacy concerns. A practical design therefore keeps the on-device part lightweight and moves the heavier analysis to a server.

Despite progress in both server-side regression detection and mobile monitoring, no existing solution combines continuous post-release monitoring on Android devices with automated, server-side detection of regressions between releases. Industry monitoring tools either provide no automated alerting or rely on fixed absolute thresholds that must be set manually for each metric, and the release-to-release comparison used in large-scale systems has not been adapted to Android, where device heterogeneity has to be taken into account. The proposed system addresses this gap by comparing consecutive releases on the server and grouping the comparison by device performance cohort.

## 2.5 Summary

This chapter reviewed existing research on mobile performance profiling, monitoring, and regression detection. Pre-release tools such as ProfileDroid, AndroStep and PowDroid offer deep system-level insights but introduce high overhead and require controlled environments, making them unsuitable for real user devices. Post-release monitoring systems such as AppInsight, PerfProbe, and Hubble avoid these drawbacks by gathering data straight from real users at minimal overhead.

The chapter also separated two kinds of metric: resource use such as CPU, memory, storage I/O, and energy, and the user-perceived responsiveness that those resources ultimately affect. It closed with regression detection, moving from manual threshold-based alerting to baseline and release-to-release comparison in large-scale systems such as FBDetect. The recurring finding is that comparing a new release against the one before it is an effective way to catch sustained degradation.

Even so, no existing solution brings together continuous, low-overhead post-release monitoring and automated server-side regression detection built for Android. This gap is what the proposed system sets out to fill.

# Chapter 3

## Methodology

This chapter describes the design of the proposed performance monitoring system. Section 3.1 lists the requirements that influenced the design choices. Section 3.2 gives a high-level overview of the architecture and shows how the components work together. Sections 3.3, 3.4 describe the details of the mobile SDK and the Backend. Section 3.4.3 discusses the regression detection methods that were considered and highlights the selected one.

### 3.1 System requirements and constraints

Since the SDK (Software Development Kit) runs on real user devices, it must be lightweight so that it doesn't affect the user experience of the host application. Moreover, the system must be robust across different Android devices. Finally, regression detection must be reliable and resilient to outliers, as production data is often noisy.

## 3.2 High-Level System Architecture

The system consists of four components: the mobile SDK (client), the Backend, the frontend, and the regression detection engine. Figure 3.1 shows how they are connected.

The mobile SDK is a library that developers integrate into their Android applications. It collects performance metrics on users' devices and sends them to the backend for further analysis.

Metrics from user devices arrive at the backend, which stores them efficiently and serves them to the frontend through REST APIs. The backend is also where user authentication and project management happen.

The frontend is a web application for developers. Through it they manage their projects, explore the collected metrics, and look over any regressions that were found. It pulls all of this from the backend through the same REST APIs and lays the data out in a readable form.

The regression detection engine continuously analyses the incoming metrics to find signs of performance degradation. If such a regression is detected, it can notify developers in a messaging app.

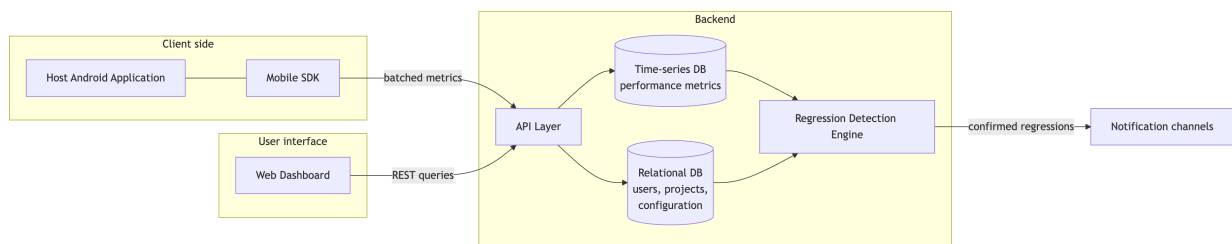


Fig. 3.1. High-Level System Architecture

## 3.3 Mobile SDK Design

### 3.3.1 Metrics Selection and Collection

The metrics were selected based on the literature review in Chapter 2. The guiding criterion was that the collected metrics should reflect how the user actually perceives the application, rather than only describe low-level system behaviour. As discussed in the review, performance metrics fall into two kinds: resource metrics, such as CPU and memory usage, that explain where a problem originates, and user-perceived metrics, such as frame time, startup time, and interaction delay, that show how strongly that problem affects the user. The two are linked, because resource pressure usually surfaces as a slower or less responsive interface. The SDK therefore collects user-perceived metrics as the primary signal of experienced performance, together with resource metrics that help explain the cause of a perceived slowdown.

These metrics are collected in two ways: as continuous metrics and as discrete events. Continuous metrics, such as CPU usage and frame time, are sampled at regular intervals. Discrete events, such as crashes, ANRs, and app startups, are recorded immediately when they happen. This approach allows the system to capture both the ongoing performance of the application and the rare but critical events that strongly affect user experience.

### 3.3.2 UI performance metrics

For UI performance, the SDK tracks frame time, FPS, startup time, interaction delays, and ANR events.

**Frame time** is one of the most important metrics for measuring UI re-

sponsiveness. It represents the time needed to render a single frame of the user interface. The target frame time depends on the display refresh rate of the device. For example, on a 60 Hz display, the frame time should stay below approximately 16.67 ms in order to keep animations and interactions smooth [18].

Frame time is measured by recording the timestamps of two consecutive frames and calculating the difference between them. If  $t_i$  and  $t_{i-1}$  are the timestamps of two consecutive frames, the frame time is calculated as follows:

$$FrameTime(ms) = \frac{t_i - t_{i-1}}{10^6} \quad (3.1)$$

**Frames per second (FPS)** is a related metric that is derived directly from the frame time. It is calculated as:

$$FPS = \frac{1000}{FrameTime(ms)} \quad (3.2)$$

**User interaction delay** measures the time between a user action (e.g. tap on a screen or a swipe) and the first visible response of the application. It is obtained by recording the timestamp of the user event and the timestamp of the first frame that was rendered after the event was processed. The delay is then calculated as:

$$InteractionDelay(ms) = t_{\text{response}} - t_{\text{event}} \quad (3.3)$$

**Startup time** shows how long the application takes to become visually responsive after it has been launched. It is defined as the time difference between the moment the application starts and the moment when the first fully rendered frame is drawn on the screen.

**ANR (Application Not Responding) events** happen when the application

is unable to handle user input for a long period of time. This is a critical metric, because ANRs directly affect user experience and can lead to the application being terminated by the operating system. ANR events are detected by monitoring the main thread. If the main thread remains blocked for more than a predefined threshold (for example, 5 seconds), an ANR event is recorded.

### 3.3.3 CPU usage

There are several ways to measure CPU usage on Android. The method selected in this work is based on the ratio between the CPU time used by the application process and the total CPU time of the system during the same interval. The main advantage of this approach is that it does not depend on the number of CPU cores or on their frequencies, which gives a normalized measure of CPU usage. This property is important, because the collected values can be compared across different devices in a fair way. On the other hand, this method may miss short-term spikes that happen between two sampling intervals. However, this is not a problem in the context of this work, because the main goal is to detect long-term performance degradation rather than short spikes of load. It is calculated as follows:

Let  $\Delta P$  be the increase in process CPU time and  $\Delta S$  be the increase in total system CPU time during the same period. Then the CPU usage of the application is calculated as:

$$CPU\% = \frac{\Delta P}{\Delta S} \cdot 100 \quad (3.4)$$

### 3.3.4 Memory usage

There are also several methods for measuring the memory used by an Android application. In this work, memory usage is measured using the Proportional Set Size (PSS) of the application process. The PSS value represents how much physical memory is effectively used by the application, including a proportional share of the memory that is shared with other processes.

The memory usage is reported in megabytes and is calculated from the PSS value in kilobytes as:

$$MemoryUsage(MB) = \frac{PSS(KB)}{1024} \quad (3.5)$$

This metric is important, because it helps to detect memory leaks and excessive memory consumption, which are frequent causes of application slowdowns and of termination by the operating system.

### 3.3.5 Data Aggregation and Transmission

The metrics collected on the device are not sent immediately. Instead, they are first stored in a local buffer and then transmitted in batches. Batching was chosen because the radio interface stays in a high-power state for several seconds after each transmission, so frequent network calls are disproportionately energy-intensive [27]. Sending metrics in batches reduces this cost and also lowers the load on the backend.

A batch is sent when one of two conditions is satisfied: either the number of metrics in the buffer reaches a predefined value  $N$ , or the time since the last transmission exceeds a time window  $W$ .

Figure 3.2 shows the sequence of operations performed by the SDK during data collection and transmission.

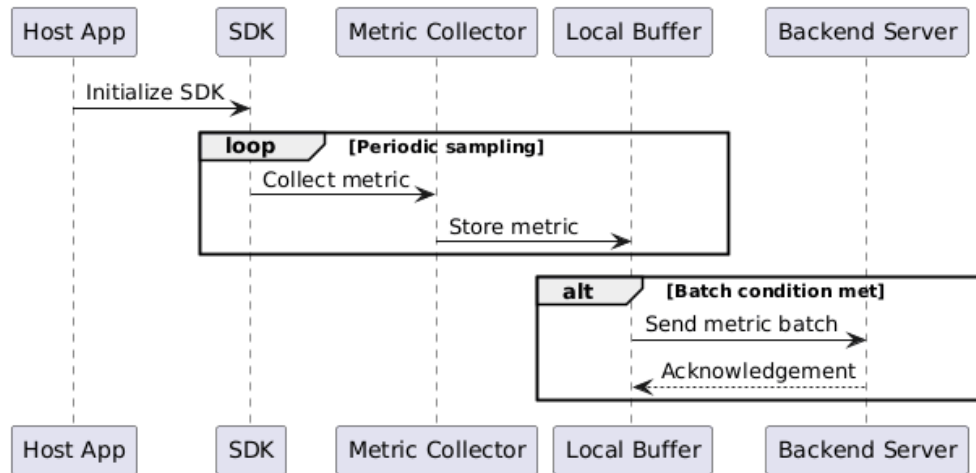


Fig. 3.2. Sequence Diagram of SDK Data Collection and Transmission

## 3.4 Backend Design

The backend has four jobs: it receives metrics from the devices, authenticates users, exposes APIs for the frontend, and runs the regression detection algorithms.

When a batch of metrics arrives from the SDK, the backend validates the request, enriches each record with device cohort information, and writes the data into the time-series database. After that, the frontend can query the stored metrics through REST endpoints, and the regression detection engine runs in the background to analyse new data as soon as it arrives.

### 3.4.1 Database Design

The backend relies on two kinds of database. A time-series database stores the performance metrics, and a relational database stores the rest: the users, their projects, and the regressions found for those projects. The time-series store

was chosen because it handles large volumes of time-stamped data efficiently [28], whereas the relational store fits the structured, interconnected records that describe accounts and their projects.

The relational side holds four entities, shown in Figure 3.3. A *user* owns one or more *projects*, each identified by its Android package name. A project, in turn, owns a catalogue of *version releases* and its detected *regressions*. The *version releases* entity records each application version observed for a project, together with when it was first seen and how many samples it has accumulated. This catalogue is refreshed periodically from the time-series store and is used to list the releases of a project. The *regressions* entity records each confirmed regression: the affected metric, screen, and device cohort, the baseline and current versions with their P95 values, and a status that marks it as open or closed. This ownership structure also enforces tenancy isolation, since every query can be scoped to the projects that belong to the requesting user.

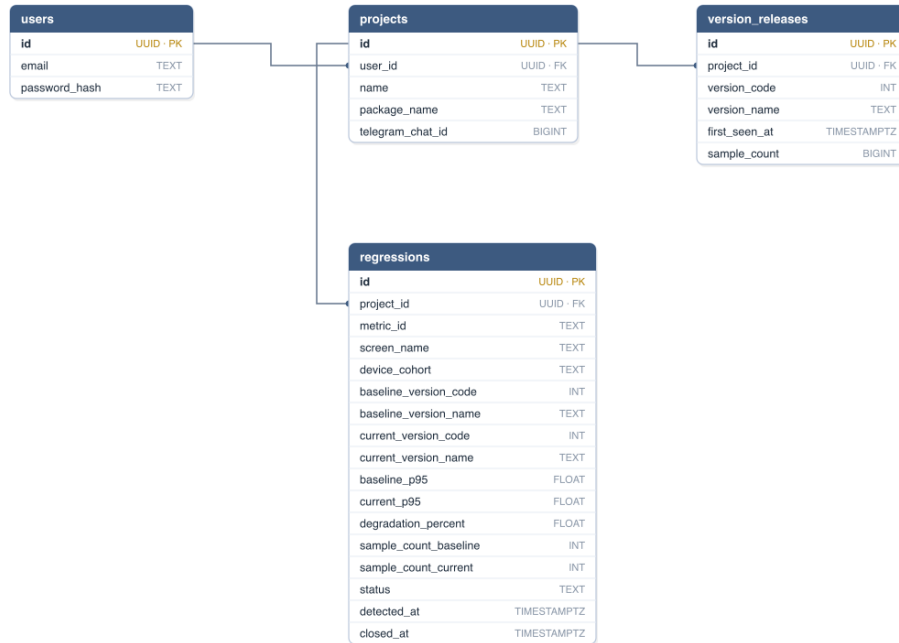


Fig. 3.3. Relational data model used by the backend

Performance metrics are kept separately in the time-series store and are not modelled as relational entities. For this purpose the system uses ClickHouse, a column-oriented database. A column-oriented layout fits the metric workload well: ingestion is append-only and write-heavy, while analysis reads only a few columns at a time and aggregates them over large ranges, such as computing a percentile per version [30]. Each metric record carries a value, its timestamp, and the dimensions along which it is later grouped, namely the project, the metric type, the screen, the device cohort, and the application version. The data is organised so that a query restricted to a recent period and to a single project and metric scans only the relevant subset, which keeps the regression-detection queries efficient as the volume of data grows.

Metrics are also retained only for a limited period of time. Older data is removed automatically, which keeps the working set relevant and reduces storage

cost. The physical layout of both stores, including table definitions and indexing, is described in Chapter 4.

### **3.4.2 Data Partitioning and Cohorts**

Before applying any detection model, the incoming data must be grouped into meaningful subsets. Comparing high-end devices directly with low-end devices could easily produce false positives, because the natural hardware limits of a low-end device might be interpreted as a software regression. To avoid this, the data is partitioned along three dimensions: the metric type (for example, CPU usage or frame time), the application screen, and the device performance cohort (Low, Medium, or High). The detection algorithms are then executed independently on each subset.

### **3.4.3 Performance Regression Detection**

The regression detection pipeline compares consecutive application releases using a release-pair P95 shift. Samples are grouped by version code rather than by a fixed time window, and each release is compared against the one that precedes it. Fixed absolute thresholds were rejected because each one must be calibrated by hand for every metric and application, and a fixed limit does not move as baseline performance shifts over time. A mean-based comparison was avoided as well, since the mean is easily pulled by outliers and copes badly with the heavy-tailed, non-normal distributions that frame time and startup time usually follow. Heavier methods such as ARIMA forecasting or machine-learning models were ruled out too, because they need labelled training data or careful tuning that a lightweight production system cannot justify.

A version is only eligible for comparison after it has accumulated a minimum number of samples. This maturity gate prevents comparisons based on a handful of early measurements, where a single noisy observation could shift the P95 far enough to cross the threshold and produce a false alert.

For each mature version group, the 95th percentile ( $P_{95}$ ) of the metric is computed. A regression is reported when the relative increase from the baseline version to the current version exceeds a configurable threshold:

$$\frac{P_{95}(\text{current}) - P_{95}(\text{baseline})}{P_{95}(\text{baseline})} > \text{Threshold} \quad (3.6)$$

Using  $P_{95}$  rather than the mean focuses detection on the worst user experiences rather than the average, and makes the result robust to skewed distributions [22]. The threshold expresses a directly interpretable criterion: a given percentage of degradation. Unlike the absolute, per-metric limits used by the threshold-based tools discussed in Chapter 2, it is a relative value that compares each release against the previous one, so a single default (15%) applies across all metrics, screens, and device cohorts. The developer therefore does not have to choose or calibrate a threshold for every metric and application. The system uses this default out of the box. This avoids the main drawback of fixed absolute thresholds, where each limit must be set and maintained by hand. The pipeline runs on production data without any labelled examples, and its core comparison is expressed as a single aggregation query, making it computationally efficient and suitable for continuous execution.

## 3.5 Summary

This chapter presented the methodology behind the proposed performance monitoring system. The design was driven by the requirements from the literature review, with an emphasis on low overhead, energy efficiency, and robustness across devices. The mobile SDK collects metrics on the user's device and sends them to the backend in batches. The backend stores them, exposes them through an API, and runs statistical regression detection on user cohorts. Together, these parts give developers automated insight into how their applications perform in production. The next chapter turns this design into a working system and describes the technical problems that came up along the way.

# Chapter 4

## Implementation

This chapter describes how the design from Chapter 3 was turned into working software.<sup>1</sup> Section 4.1 covers the mobile SDK, from initialization to the individual collectors and the buffering and transmission pipeline. Section 4.2 describes the Backend, focusing on user management and the ingestion path. Section 4.3 explains the regression detection pipeline in detail. Section 4.4 then walks through the frontend web application and its main features.

### 4.1 Mobile SDK Implementation

The mobile part of the monitoring system was implemented as an Android library written in Kotlin. Kotlin was chosen because it is the official, recommended language for Android development [29].

The SDK is distributed through a Maven repository, which allows developers to add it to their project easily by adding a dependency to their `build.gradle` file.

---

<sup>1</sup>The source code is available at <https://github.com/nailka/PerfX>.

### 4.1.1 Initialization and Lifecycle Management

One of the requirements for the SDK was to make the integration as simple as possible for developers. To achieve this, the entire setup is done with a single call to the `PerfX.initialize()` method inside the application class. The developer only needs to provide the application context and a unique project ID.

Initialization runs a short sequence of steps. The `AppInfoProvider` collects device and app metadata (OS version, device model, app version). The SDK then opens the local Room database (`MetricDatabase`), starts the background sync manager, and creates the metric collectors. Finally, the SDK registers an `Application.ActivityLifecycleCallbacks` listener so it can follow the active screen and pause network sync while the app sits in the background, which saves battery.

#### Listing 4.1: SDK Initialization

```
1 fun initialize(application: Application, projectId: String) {  
2     if (isRunning) return  
3  
4     appInfo = AppInfoProvider().get(application, projectId)  
5     db = MetricDatabase.getInstance(application)  
6     SyncManager.startPeriodicSync(application)  
7  
8     // ... Collector initialization (described below)  
9  
10    registerActivityCallback(application)  
11    isRunning = true  
12 }
```

### 4.1.2 Performance Metrics Collectors

Each metric collector extends a common abstract class called `PerformanceCollector`. This class defines a single `collect()` method that returns a Kotlin Flow of Metric objects, which allows the SDK to treat all collectors in a uniform way.

**Listing 4.2:** PerformanceCollector Abstract Class

```
1 internal abstract class PerformanceCollector {  
2     abstract fun collect(intervalMs: Long): Flow<Metric>  
3     abstract fun stop()  
4 }
```

All individual metric streams are merged into a single Flow inside the `MetricsRepository` class. This design makes it easy to add new collectors in the future without changing the rest of the pipeline.

#### 1. Memory Collector

The `RamCollector` runs on a background thread and measures the memory footprint of the application at a fixed sampling interval. It uses two Android APIs: `Debug.getMemoryInfo()` to get the total PSS (Proportional Set Size) of the process, and the Java Runtime to calculate the current Java heap usage. The Java heap usage is calculated as the difference between the total allocated heap and the currently free heap. This value is what gets emitted as the metric, because it reflects the memory actually used by the application code.

**Listing 4.3:** Memory Collector Implementation

```
1 val memoryInfo = Debug.MemoryInfo()  
2 Debug.getMemoryInfo(memoryInfo)  
3  
4 val runtime = Runtime.getRuntime()  
5 val javaHeapUsedMb = (runtime.totalMemory() - runtime.freeMemory
```

```

    )) / (1024.0 * 1024.0)
6  val pssMb = memoryInfo.totalPss / 1024.0
7
8  emit(Metric.MemoryUsage(timestamp = System.currentTimeMillis(),
    value = javaHeapUsedMb))

```

## 2. CPU Collector

Because modern Android versions restrict direct access to the `/proc/stat` file for security reasons, the `CpuCollector` uses the `android.os.Process` API instead. From this API it reads the accumulated CPU time of the application process at two points, one sampling interval apart. The CPU percentage is the CPU time used during that interval divided by the wall-clock length of the interval:

$$CPU\% = \frac{\Delta CPU_{process}}{\Delta t_{wall}} \cdot 100 \quad (4.1)$$

This ratio expresses how much CPU time the application used relative to real time, so the values stay comparable across devices with different core counts.

## 3. Frame Time Collector

Frame rendering time is captured through a `Choreographer.FrameCallback` registered on the main thread. Android invokes this callback just before each new frame is drawn and passes the frame timestamp in nanoseconds. Subtracting the timestamps of two consecutive callbacks gives the raw frame time. Individual frame times are accumulated over a fixed interval and then averaged before being emitted as a single metric. This aggregation reduces the number of data points that need to be stored and transmitted, while still capturing the overall rendering performance of the window.

**Listing 4.4:** Frame Time Collector: core measurement

```
1 val callback = Choreographer.FrameCallback { frameTimeNs ->
2     if (lastFrameTimeNs != 0L) {
3         val deltaMs = (frameTimeNs - lastFrameTimeNs) / 1_000_000
4             .0
5         if (deltaMs >= 0.0) {
6             frameCount++
7             totalFrameTimeMs += deltaMs
8         }
9     }
10    lastFrameTimeNs = frameTimeNs
11    Choreographer.getInstance().postFrameCallback(frameCallback)
12 }
13 // Emitted every intervalMs
14 val avgFrameTime = if (frameCount > 0) totalFrameTimeMs /
15     frameCount else 0.0
16 emit(Metric.FrameTime(timestamp = System.currentTimeMillis(),
17     value = avgFrameTime))
```

#### 4. Startup Time Collector

Startup time is measured by recording the timestamp at the moment the application process starts and comparing it to the timestamp of the first fully drawn frame. The SDK uses the `reportFullyDrawn()` API to detect when the first screen has finished rendering. The difference between these two timestamps is reported as the startup time.

#### 5. Interaction Delay Collector

User interaction delay is measured by intercepting touch events on the root view of each activity. When a touch event is received, its timestamp is recorded. The delay is then calculated as the time between the touch event and the first frame

rendered after the event was processed, which is detected using the Choreographer callback.

## 6. ANR Detector

The SDK detects ANR events using a watchdog pattern. A background thread posts a short task to the main thread's Handler and waits for it. If the task does not run within the timeout (5 seconds by default, matching the official Android documentation [20]), the watchdog treats the main thread as blocked and records an ANR. The approach needs no special permissions and works on every Android version.

**Listing 4.5:** ANR Watchdog: detection logic

```
1 val mHandler = Handler(Looper.getMainLooper())
2 var mainThreadResponded = false
3
4 mHandler.post { mainThreadResponded = true }
5 Thread.sleep(ANR_TIMEOUT_MS)
6
7 if (!mainThreadResponded) {
8     emit(Metric.Anr(timestamp = System.currentTimeMillis()))
9 }
```

### 4.1.3 Data Buffering and Network Transmission

The SDK uses a two-tier buffering approach to make sure that no metrics are lost if the application crashes or the network is temporarily unavailable.

#### Tier 1: In-Memory Buffer

As metrics arrive from the various collectors, they are first stored in a ConcurrentLinkedQueue. This structure was chosen because several collectors

write at the same time from different background threads, and a plain list could lead to race conditions. A `ConcurrentLinkedQueue` handles concurrent access safely without explicit locks, which keeps the overhead low.

When the number of items in the memory buffer reaches a predefined limit (`MEMORY_LIMIT`), the buffer is flushed to disk.

#### Listing 4.6: In-Memory Buffering

```
1 flow.collect { metric ->
2     memoryBuffer.add(metric)
3     memoryBufferSize += 1
4
5     if (memoryBufferSize >= Constants.MEMORY_LIMIT) {
6         flushToDisk(db, context)
7         memoryBufferSize = 0
8     }
9 }
```

#### Tier 2: Disk Storage and Network Synchronization

On a flush, the metrics move to a local SQLite database managed by the Room library. Each record is stored together with the name of the screen that was visible at the time, which the lifecycle callback registered during initialization keeps track of.

Once the database holds `DISK_LIMIT` records, the SDK starts a network synchronization immediately. A background job also runs every 15 minutes and uploads whatever is left, even if that limit has not been reached, so metrics are still sent when the application is used only rarely.

The network layer uses Retrofit and OkHttp. The SDK reads a batch of records from the database, serializes it to JSON with Kotlinx Serialization, and sends it to the backend in a single HTTP POST request. On success, the uploaded

records are deleted from the local database. On failure, they stay on disk and are retried at the next synchronization.

**Listing 4.7: Network Synchronization**

```
1 val batchApi = BatchApi(  
2     appInfo = PerfX.appInfo ,  
3     metrics = batch.map { it.mapToApi() }  
4 )  
5  
6 val response = NetworkClient.api.uploadBatch(batchApi)  
7  
8 if (response.isSuccessful) {  
9     db.metricDao().deleteMetrics(batch.map { it.id })  
10 }
```

## 4.2 Backend Implementation

The backend was built in Kotlin with the Ktor framework. Ktor is lightweight and asynchronous, which fits a REST API well. It serves each request on a coroutine, so it can handle many connections at once without blocking threads.

It relies on two databases: PostgreSQL for relational data such as user accounts and project metadata, and ClickHouse for the time-series performance metrics.

### 4.2.1 User and Project Management

PostgreSQL holds the user accounts, projects, version releases, and detected regressions, following the relational data model shown in Chapter 3 (Figure 3.3).

Passwords are hashed with BCrypt before they are stored. At login, the server checks the supplied password against the stored hash, and on success it issues a JSON Web Token (JWT). Every later request to the API must carry this token, which keeps each user's data accessible only to that user.

To create a project, a developer submits a name and an Android package name, and the server rejects either one if it is already taken. Once the project passes this check, it is saved and linked to the developer's account. A background job periodically refreshes the `version_releases` catalogue from ClickHouse, recording each observed version code together with its first-seen time and accumulated sample count. The dashboard uses this catalogue to list the releases of a project.

### 4.2.2 Data Ingestion

The data ingestion endpoint is the most performance-critical part of the backend, because the mobile SDKs from many users can send large numbers of metrics at the same time. Storing this data in PostgreSQL would not be efficient enough for this workload. For this reason, all performance metrics are stored in ClickHouse, which is a column-oriented database optimized for fast batch insertions and analytical range queries over time [30].

The `metric_records` table was designed to support the queries needed by the regression detection pipeline efficiently. The table uses the MergeTree engine, which is optimized for fast insertions and range scans. The data is partitioned by date so that queries over a recent time window only scan the relevant partitions. The primary sort key is `(project_id, metric_id, ts)`, which makes range queries over time for a specific project and metric very fast.

**Listing 4.8:** ClickHouse metric\_records table schema

```
1 CREATE TABLE IF NOT EXISTS metrics.metric_records
2 (
3     project_id      String ,
4     package_name    String ,
5     ts              DateTime64(3, 'UTC') ,
6     metric_id       LowCardinality(String) ,
7     screen_name     LowCardinality(String) ,
8     value           Float64 ,
9     app_version     LowCardinality(String) ,
10    os_version       LowCardinality(String) ,
11    device_model     LowCardinality(String) ,
12    device_cohort    LowCardinality(String) DEFAULT 'Unknown' ,
13    received_at      DateTime DEFAULT now()
14 )
15 ENGINE = MergeTree
16 PARTITION BY toDate(ts)
17 ORDER BY (project_id , metric_id , ts);
```

When the mobile SDK sends a POST request to the `/ingest` endpoint, the server receives a batch of metrics together with the application and device metadata. Instead of inserting each metric individually, the ClickHouseClient formats the entire batch as a single `JSONEachRow` payload and executes one bulk `INSERT` query. Before saving each record, the server also assigns the device cohort (Low, Medium, or High) based on the hardware metadata sent by the SDK. Batching the inserts this way keeps pressure on the database low, so the server can absorb many simultaneous requests before it turns into a bottleneck.

## 4.3 Performance Regression Detection

The regression detection pipeline described in Chapter 3 was split into three stages: device cohort classification on the Ktor backend, per-version P95 aggregation using ClickHouse queries, and release-pair comparison in a Python service.

The first stage assigns the device cohort and runs on the backend. The SDK includes the total RAM and the CPU core count of the device in every batch it uploads. From these values the backend determines the device cohort. It reads this metadata from each incoming request, derives the cohort, and stores it with every metric before writing the data to ClickHouse.

**Listing 4.9:** Device cohort classification

```
1 fun calculateDeviceCohort(totalRamGb: Double, cpuCores: Int):  
   String {  
2     return when {  
3       totalRamGb <= 3.0 -> "Low"  
4       totalRamGb <= 6.0 && cpuCores <= 8 -> "Medium"  
5       else -> "High"  
6     }  
7 }
```

In the second stage, the pipeline computes the P95 of each version group inside ClickHouse with the built-in quantile function. The query groups records by project, metric, screen, device cohort, and version code, and discards any version group that has not yet collected enough samples to count as mature. The query returns a single P95 per (metric, screen, cohort, version) combination, ordered by version code so that consecutive releases can be paired up.

**Listing 4.10:** ClickHouse query for per-version P95 values

```
1 SELECT
2     project_id ,
3     metric_id ,
4     screen_name ,
5     device_cohort ,
6     version_code ,
7     version_name ,
8     quantile(0.95)(value) AS p95 ,
9     count()              AS cnt
10 FROM metric_records
11 GROUP BY
12     project_id , metric_id , screen_name ,
13     device_cohort , version_code , version_name
14 HAVING cnt >= {min_samples}
15 ORDER BY
16     project_id , metric_id , screen_name ,
17     device_cohort , version_code
```

In the final stage, once ClickHouse has returned the mature versions, the detector iterates over each group and compares consecutive version pairs. For every pair, it computes the relative P95 shift and evaluates it against the configured threshold. If the shift exceeds the threshold, the corresponding regression is recorded in the regressions table in PostgreSQL.

**Listing 4.11:** Release-pair comparison in the detection service

```
1 for i in range(len(versions) - 1):
2     baseline_vc , baseline_vn , baseline_p95 , baseline_cnt =
3         versions[i]
4     current_vc , current_vn , current_p95 , current_cnt =
5         versions[i + 1]
```

```
4
5     if baseline_p95 <= 0:
6         continue
7
8     delta = (current_p95 - baseline_p95) / baseline_p95
9
10    if delta > threshold:
11        upsert_regression(pg_conn, ..., delta)
```

The threshold is a single global value of 15%, defined in the detector configuration and applied uniformly to every group. The system does not store per-project or per-metric thresholds. Using one relative threshold avoids the manual per-metric calibration required by the absolute threshold-based tools discussed in Chapter 2.

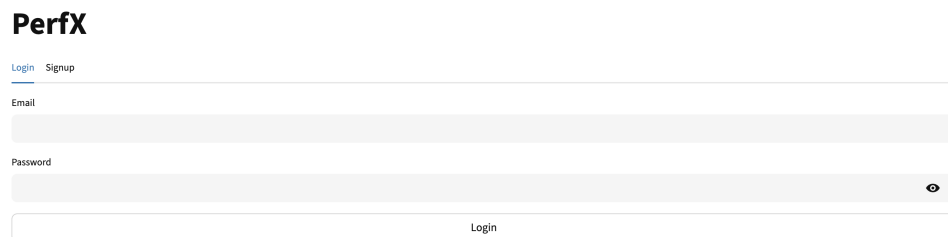
Once a regression has been flagged, it is written to the `regressions` table in PostgreSQL together with the metric type, affected screen, device cohort, the baseline and current P95 values, and the degradation percentage. Detected regressions appear on the dashboard, where developers can inspect and investigate active issues.

## 4.4 Frontend Implementation

The frontend was built with Streamlit, a Python framework for interactive web apps. Streamlit made it possible to build a data-driven interface quickly, without maintaining a separate frontend codebase. Plotly renders the charts.

### 4.4.1 User Authorization

Users sign up and log in with an email and password. The backend returns a JWT on a successful login, and the dashboard keeps it in the browser session. From then on, every call to the backend API carries that token as proof of identity.



The image shows a web form for logging in. At the top left is the text 'PerfX'. Below it are two links: 'Login' (underlined) and 'Signup'. There are two input fields: 'Email' and 'Password'. The 'Password' field has a small eye icon on the right side. At the bottom of the form is a button labeled 'Login'.

Fig. 4.1. Login page of the dashboard

### 4.4.2 Project Management

Once logged in, a user sees their existing projects and can add or remove projects from the same page. When a new project is created, the dashboard displays the unique project ID that the developer needs to use when initializing the mobile SDK in their Android application.

The screenshot shows a web application interface for managing Android projects. On the left is a sidebar with a 'User' section (email: elon@tesla.com, Logout button), a 'Project' dropdown (currently showing 'Demo Project'), and a 'Navigation' menu with links to Dashboard, Metrics, Analysis, and Projects (which is highlighted). The main content area is titled 'Projects' with the subtitle 'Create and manage your Android applications'. It features a 'Create new project' form with fields for 'Project name' (filled with 'Test Project'), 'Android App ID' (filled with 'ru.ndevelop.test'), and a 'Create project' button. Below the form is a green success message: 'Project created successfully'. Underneath, it says 'Use this identifier in your Android SDK:' followed by the ID '1d1b0ac4-29ae-408a-8a25-a21776e2fed6'. At the bottom, there is a section 'Existing projects' containing a table with two rows of project data.

| id                                     | name         | package_name     |
|----------------------------------------|--------------|------------------|
| 0 bda099a-a3b0-4a54-a5e7-eeee1ee5198f  | Demo Project | ru.ndevelop.demo |
| 1 1d1b0ac4-29ae-408a-8a25-a21776e2fed6 | Test Project | ru.ndevelop.test |

Fig. 4.2. Project list page

### 4.4.3 Metrics Explorer

The metrics explorer presents the raw data in a table, where each measurement appears with its timestamp, value, and metadata. The table can be exported to CSV for deeper analysis in tools such as Excel or Jupyter Notebook. This helps with debugging and with one-off analyses.

The screenshot shows the 'Metrics Explorer' interface. At the top, it says 'Project: PerfX Demo'. Below this is a form with three input fields: 'Metric ID' (filled with 'frameTime'), 'Screen Name' (filled with 'compose/ui\_responsiveness'), and 'Max rows' (filled with '5000' and has minus/plus icons). At the bottom of the form is a 'Load metrics' button.

Fig. 4.3. Metrics Explorer: filter selection

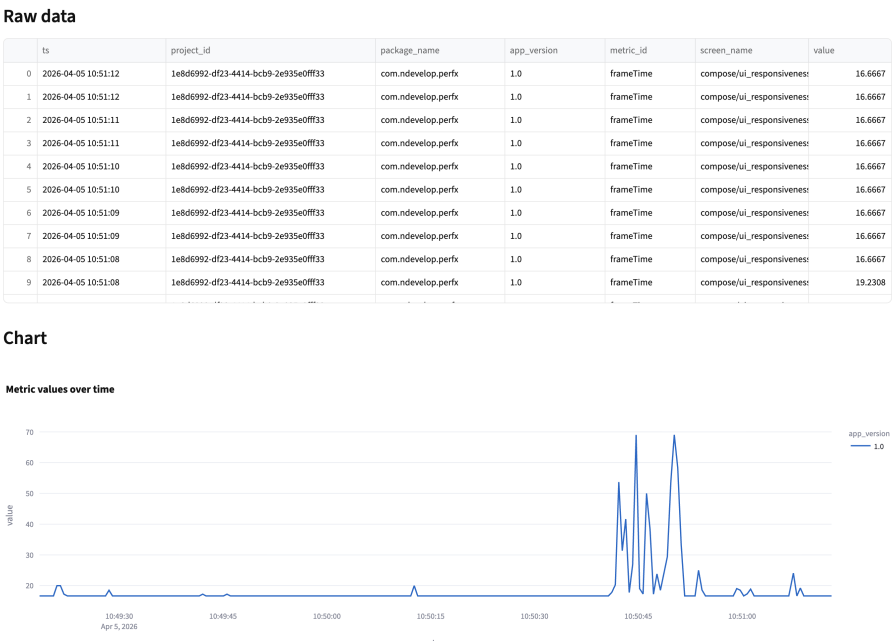
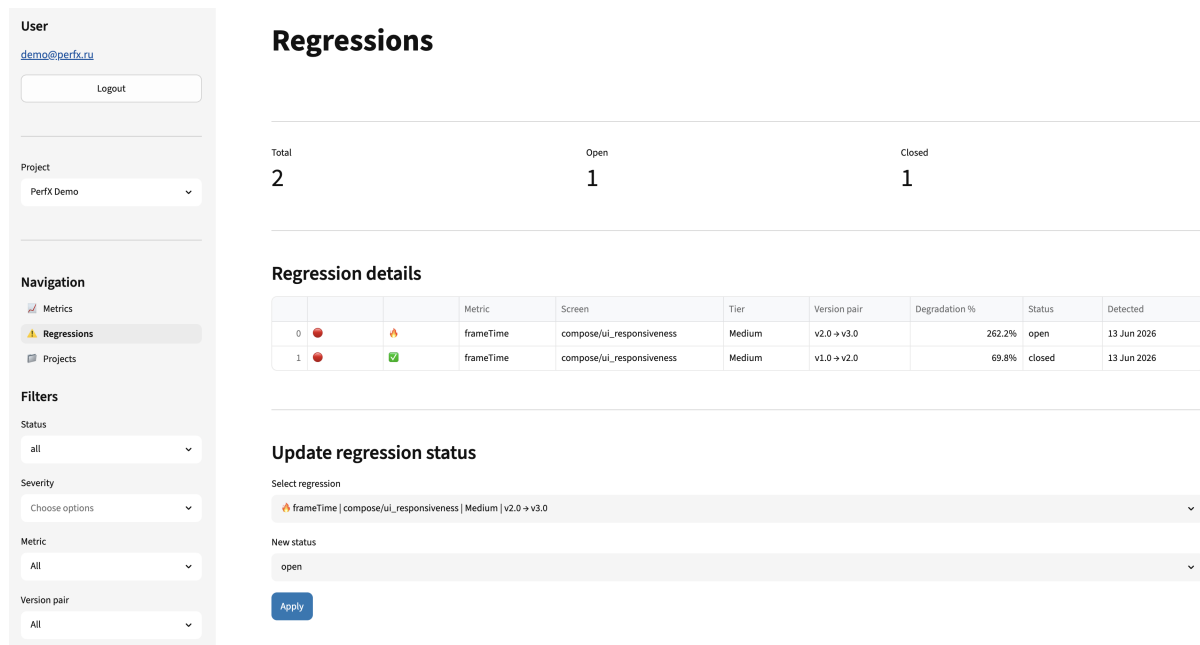


Fig. 4.4. Metrics Explorer: frame time chart with a detected regression

4.4.4 Regressions

The regressions page lists every regression that the detection pipeline has reported for the selected project. Each row shows the affected metric, screen, and device cohort, the version pair that was compared, the degradation, and whether the regression is open or closed. A summary at the top of the page counts how many regressions are open and how many are closed. From the same page a developer can filter the list and close a regression once it has been handled.



**User**  
demo@perfx.ru  
Logout

**Project**  
PerfX Demo

**Navigation**  
Metrics  
Regressions  
Projects

**Filters**  
Status: all  
Severity: Choose options  
Metric: All  
Version pair: All

## Regressions

| Total | Open | Closed |
|-------|------|--------|
| 2     | 1    | 1      |

### Regression details

|   |   |   | Metric    | Screen                    | Tier   | Version pair | Degradation % | Status | Detected    |
|---|---|---|-----------|---------------------------|--------|--------------|---------------|--------|-------------|
| 0 | ● | 🔥 | frameTime | compose/ui_responsiveness | Medium | v2.0 + v3.0  | 262.2%        | open   | 13 Jun 2026 |
| 1 | ● | ✅ | frameTime | compose/ui_responsiveness | Medium | v1.0 + v2.0  | 69.8%         | closed | 13 Jun 2026 |

### Update regression status

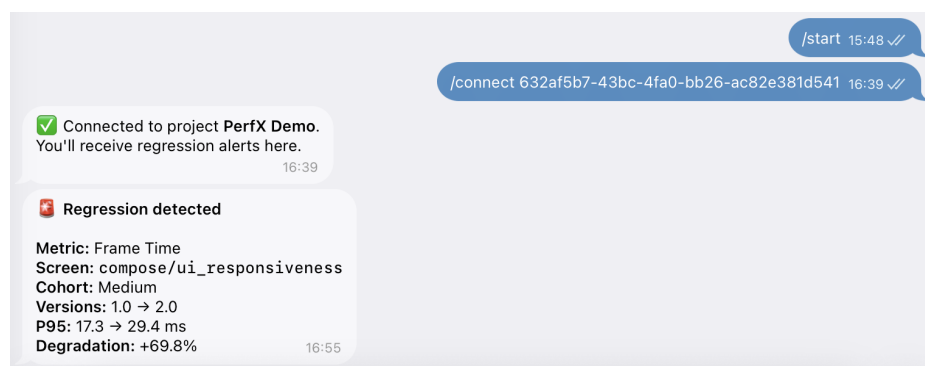
Select regression  
🔥 frameTime | compose/ui\_responsiveness | Medium | v2.0 + v3.0

New status  
open

Apply

Fig. 4.5. Regressions page of the dashboard

Developers can also receive regression alerts in messenger. Each message reports the affected metric, screen, and device cohort, the compared version pair, and the magnitude of the degradation.



/start 15:48 ✓✓

/connect 632af5b7-43bc-4fa0-bb26-ac82e381d541 16:39 ✓✓

✅ Connected to project **PerfX Demo**.  
You'll receive regression alerts here.  
16:39

❌ **Regression detected**

Metric: Frame Time  
Screen: compose/ui\_responsiveness  
Cohort: Medium  
Versions: 1.0 → 2.0  
P95: 17.3 → 29.4 ms  
Degradation: +69.8%  
16:55

Fig. 4.6. Regression alert delivered to a messaging app

## 4.5 Summary

This chapter described the implementation of all four system components. The mobile SDK was built as a Kotlin Android library that collects six performance metrics, buffers the data in a two-tier storage layer, and transmits it to the backend in batches. The Backend receives the data and stores it in two separate databases: ClickHouse for time-series metrics and PostgreSQL for user and project metadata. The regression detection pipeline was implemented as a Python service that computes per-version P95 values using ClickHouse aggregation queries and flags regressions when the relative P95 shift between consecutive releases exceeds the configured threshold. Finally, the frontend was built with Streamlit and lets developers explore the collected metrics and review detected regressions. The next chapter evaluates how well this system works in practice.

# Chapter 5

## Evaluation and Discussion

This chapter evaluates the performance monitoring system presented in Chapters 3 and 4. The evaluation addresses three research questions: how much overhead the SDK adds to a host application, how the regression detection pipeline identifies regressions introduced by application updates, and how accurately it does so. Section 5.1 describes the validation setup. Section 5.2 reports the overhead introduced by the SDK. Section 5.3 explains why the validity of the collected metrics rests on the platform APIs. Section 5.4 evaluates the regression detection pipeline using injected regressions. Section 5.5 interprets the results in relation to the research questions, and Section 5.6 discusses the limitations of the evaluation.

### 5.1 Validation Setup

The evaluation was divided into two parts: a measurement of the overhead added by the SDK (Section 5.2), and an evaluation of the regression detection pipeline using injected regressions (Section 5.4).

### 5.1.1 Test Devices

The experiments were carried out on one physical Android device and three emulator instances, each configured with a different amount of RAM and a different number of CPU cores, so that the results cover all three performance cohorts defined in Section 3.4.3. Table 5.1 lists each device together with its hardware characteristics, its Android version, and the cohort assigned by the backend.

TABLE 5.1  
Test Devices Used in the Evaluation

| Device                   | RAM (GB) | CPU cores | Android version | Cohort |
|--------------------------|----------|-----------|-----------------|--------|
| Emulator (Low cohort)    | 2        | 2         | 11              | Low    |
| Emulator (Medium cohort) | 4        | 4         | 13              | Medium |
| Xiaomi Redmi Note 9 Pro  | 6        | 8         | 15              | Medium |
| Emulator (High cohort)   | 8        | 4         | 14              | High   |

### 5.1.2 Test Application

Two separate applications were used across the evaluation, each serving a different purpose.

The overhead experiments used Cheddar [31], an open-source Android client for the Hacker News website. It represents a realistic application workload, combining network requests, list rendering, and in-app navigation.

The regression detection experiments used a dedicated demo application developed as part of this project. It contains three screens that exercise different metrics, covering three regression types: increased CPU load, a memory leak, and UI thread blocking. Each experiment compared two builds of the application assigned different version codes, so the detector treated them as consecutive

releases.

## 5.2 SDK Overhead Evaluation

SDK overhead was measured by running the same scenario on Cheddar in two configurations: with the SDK enabled and without it. CPU usage and memory usage were recorded during the 90-second run, and cold-start time was measured separately as the mean of ten launches. Table 5.2 and Table 5.3 report the results for each test device.

TABLE 5.2  
SDK Overhead Measured on the Test Devices

| Device                   | CPU usage (%) |          |          | Memory (MB) |          |          |
|--------------------------|---------------|----------|----------|-------------|----------|----------|
|                          | Without SDK   | With SDK | Overhead | Without SDK | With SDK | Overhead |
| Xiaomi Redmi Note 9 Pro  | 94.6          | 95.4     | +0.8     | 220.1       | 222.0    | +1.9     |
| Emulator (Low cohort)    | 39.9          | 44.4     | +4.5     | 149.8       | 153.5    | +3.7     |
| Emulator (Medium cohort) | 19.6          | 24.8     | +5.2     | 131.1       | 136.2    | +5.1     |
| Emulator (High cohort)   | 32.2          | 33.9     | +1.8     | 131.4       | 136.7    | +5.4     |

TABLE 5.3  
Cold-Start Time Overhead Measured on the Test Devices (mean of ten launches)

| Device                   | Without SDK (ms) | With SDK (ms) | Overhead (ms) | Overhead (%) |
|--------------------------|------------------|---------------|---------------|--------------|
| Xiaomi Redmi Note 9 Pro  | 1484             | 1559          | +75           | +5.1         |
| Emulator (Low cohort)    | 1081             | 1152          | +71           | +6.6         |
| Emulator (Medium cohort) | 609              | 677           | +68           | +11.2        |
| Emulator (High cohort)   | 664              | 693           | +29           | +4.4         |

On the physical Redmi Note 9 Pro, the SDK added 0.8 percentage points of CPU usage and 1.9 MB of resident memory during the 90 second scrolling scenario. The cold-start overhead was 75 ms on average, a one-time cost paid at each process launch. The CPU values can exceed 100 % because they aggregate

process time across all eight cores. The baseline of approximately 95 % reflects a multi-core workload, not a saturated single core.

On the emulators the CPU overhead was noticeably higher in absolute terms: 4.5 percentage points on the Low-cohort emulator, 5.2 on the Medium-cohort emulator, and 1.8 on the High-cohort emulator. This is expected because the baseline CPU activity on emulators is much lower than on the physical device, so the SDK’s background threads and periodic sampling make up a larger share of the total. The memory overhead was more consistent across all four devices, ranging from 1.9 MB on the physical device to 5.4 MB on the High-cohort emulator. Cold-start overhead was similar in absolute terms across all devices, between 29 ms and 75 ms.

Figure 5.1 shows the per-device overhead charts produced by the measurement notebook.

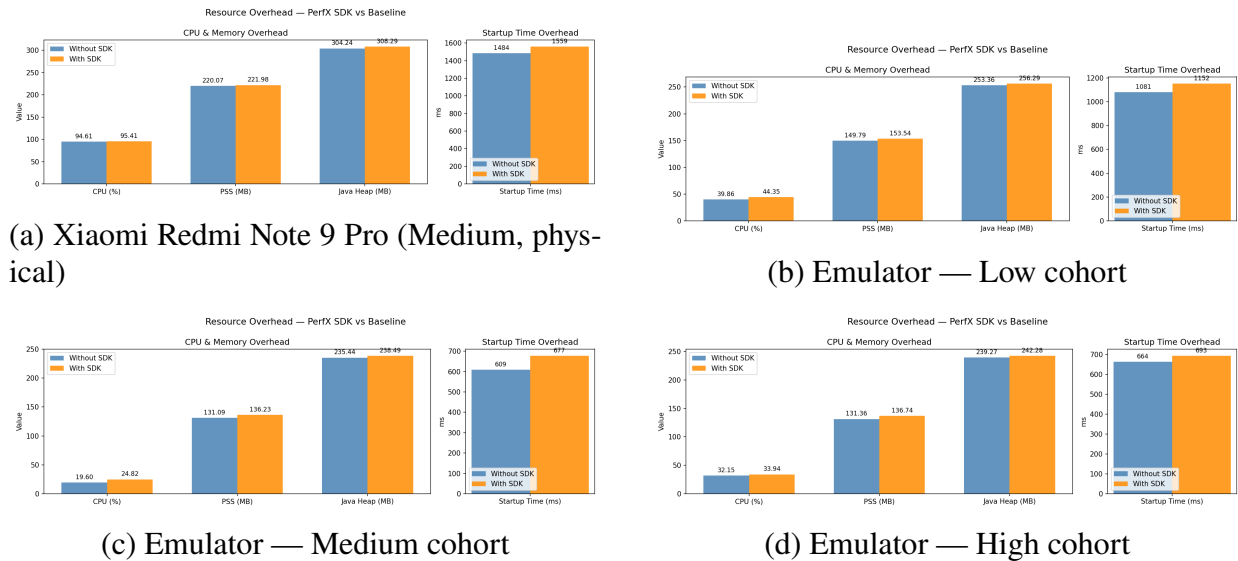


Fig. 5.1. SDK overhead charts for all four test devices. Each chart shows mean CPU usage, PSS memory, and cold-start time without and with the SDK enabled.

The measured CPU overhead on the physical device is lower than the approximately 6 % reported for the Panappicon tracing system [14]. Together with

the consistent memory overhead and the moderate one-time cold-start cost, these results support the low-overhead requirement defined in Section 3.1 across all tested device cohorts.

## **5.3 Metric Validity**

Each metric is collected using the authoritative Android platform API for that measurement, as described in Section 4.1. For this reason, a separate metric-accuracy experiment against external reference tools was not carried out: it would compare one platform API against another rather than testing the SDK itself, and any observed deviation would reflect differences in sampling granularity or timing between tools, not errors introduced by the SDK. The correctness of the SDK's measurements therefore rests on the correctness of the Android platform APIs, which are maintained by the Android Open Source Project and used in production profiling tooling. The evaluation accordingly focuses on detection accuracy (Section 5.4), not on re-measuring the platform APIs.

## **5.4 Regression Detection Evaluation**

This section evaluates the ability of the regression detection pipeline to correctly identify performance regressions and to produce no alerts in their absence. Two builds of the demo application were compared for each experiment: a baseline build with no regression and a second build with a known regression introduced, as described in Section 5.1.

### 5.4.1 Experiment Setup

Each build was launched on the target device and run for one minute, during which the SDK forwarded metric samples to the backend. The detector then compared the P95 of each metric between the two version codes, grouped by metric, screen, and device cohort. A regression was flagged when the relative P95 shift exceeded 15%.

**Experiment matrix.** Three regression types (CPU load, memory leak, UI thread blocking) were evaluated at three intensity levels (low, medium, high) on three device cohorts (Low, Medium, High), giving 27 experiment cells in total.

**Control runs.** Five runs per cohort with no injected regression were carried out to measure the false-positive rate. These results are reported in Section 5.4.4.

### 5.4.2 Regression Injection

A CPU regression was simulated by launching one or more background coroutines on `Dispatchers.Default`, each running a tight prime-number sieve in a loop. The number of concurrent workers was one, two, and four for low, medium, and high intensity. A 10 ms cooperative yield was added to each iteration so that the SDK upload coroutines were not completely starved.

A memory regression was simulated by allocating 256 KB byte arrays at regular intervals and retaining all of them in a list, preventing the garbage collector from reclaiming them. The allocation rate was one, two, and three chunks every 200 ms for low, medium, and high intensity.

A UI thread regression was simulated by calling `Thread.sleep()` inside a `withFrameMillis` callback on the main thread, blocking it for a fixed duration on every rendered frame. The block durations were 15 ms, 40 ms, and 90 ms per

frame for low, medium, and high intensity respectively.

### 5.4.3 Detection of Injected Regressions

Table 5.4 reports the baseline and current P95 values, the relative P95 shift, and the detection outcome for each of the 27 cells. The detection rate aggregated across all 30 runs is shown in Figure 5.2.

TABLE 5.4

P95 Shift and Detection Outcome for the Injected Regressions (Release-Pair Comparison, 15% Threshold)

| Metric          | Intensity | Cohort | Baseline | Current | $\Delta$ (%) | Detected         |
|-----------------|-----------|--------|----------|---------|--------------|------------------|
| CPU usage (%)   | Low       | Low    | 6.29     | 7.15    | 13.6         | No               |
| CPU usage (%)   | Low       | Medium | 8.31     | 9.55    | 15.1         | Partial (2 of 4) |
| CPU usage (%)   | Low       | High   | 10.22    | 12.97   | 27.0         | Yes              |
| CPU usage (%)   | Medium    | Low    | 9.49     | 17.20   | 81.3         | Yes              |
| CPU usage (%)   | Medium    | Medium | 3.79     | 5.16    | 36.0         | Yes              |
| CPU usage (%)   | Medium    | High   | 10.18    | 20.24   | 98.8         | Yes              |
| CPU usage (%)   | High      | Low    | 9.40     | 137.53  | 1363.0       | Yes              |
| CPU usage (%)   | High      | Medium | 9.98     | 132.77  | 1230.3       | Yes              |
| CPU usage (%)   | High      | High   | 10.56    | 170.06  | 1510.8       | Yes              |
| Memory (MB)     | Low       | Low    | 72.88    | 87.99   | 20.7         | Yes              |
| Memory (MB)     | Low       | Medium | 71.04    | 86.19   | 21.3         | Yes              |
| Memory (MB)     | Low       | High   | 68.94    | 84.13   | 22.0         | Yes              |
| Memory (MB)     | Medium    | Low    | 72.86    | 152.23  | 108.9        | Yes              |
| Memory (MB)     | Medium    | Medium | 71.12    | 150.66  | 111.8        | Yes              |
| Memory (MB)     | Medium    | High   | 68.96    | 148.90  | 115.9        | Yes              |
| Memory (MB)     | High      | Low    | 72.83    | 202.09  | 177.5        | Yes              |
| Memory (MB)     | High      | Medium | 71.63    | 200.41  | 179.8        | Yes              |
| Memory (MB)     | High      | High   | 69.09    | 198.35  | 187.1        | Yes              |
| Frame time (ms) | Low       | Low    | 16.67    | 25.83   | 55.0         | Yes              |
| Frame time (ms) | Low       | Medium | 16.67    | 20.00   | 20.0         | Yes              |
| Frame time (ms) | Low       | High   | 19.23    | 29.41   | 52.9         | Yes              |
| Frame time (ms) | Medium    | Low    | 16.67    | 50.00   | 200.0        | Yes              |
| Frame time (ms) | Medium    | Medium | 16.67    | 45.46   | 172.7        | Yes              |
| Frame time (ms) | Medium    | High   | 17.26    | 51.67   | 199.3        | Yes              |
| Frame time (ms) | High      | Low    | 16.67    | 100.00  | 500.0        | Yes              |
| Frame time (ms) | High      | Medium | 16.67    | 100.00  | 500.0        | Yes              |
| Frame time (ms) | High      | High   | 17.27    | 106.67  | 517.5        | Yes              |

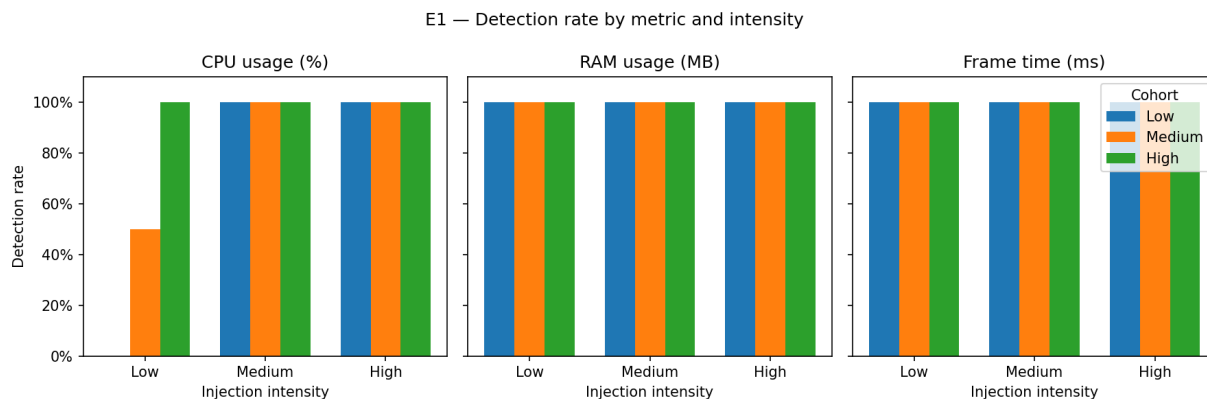


Fig. 5.2. Detection rate by metric, injection intensity, and device cohort. Each bar shows the fraction of runs in that cell that the detector flagged as a regression at the 15% threshold.

The detector flagged 27 of the 30 runs (90%). All 27 cells with intensity at medium or high were detected on every cohort, including the very large P95 shifts produced by the high-intensity injections. The three missed runs all fall into the low-intensity CPU configuration: the Low-cohort run produced a P95 shift of 13.6%, just below the 15% threshold, and the Medium-cohort cell was at the threshold border, with two of four runs detected and two not. The High cohort still detected the same configuration at a P95 shift of 27.0%, because the baseline CPU usage on that emulator was slightly higher and the same injection produced a larger relative shift.

The remaining 24 cells were detected with margin: the smallest relative shift among detected cells was 20.0% (UI low on Medium), and most cells exceeded the threshold by an order of magnitude. The detector therefore reliably flags injections whose relative effect on the P95 value is at least roughly twice the configured threshold and behaves less reliably for injections whose effect is close to or below the threshold.

### 5.4.4 Threshold Sensitivity

The 15% threshold used above is one configuration choice. To show the trade-off between detection rate and false positives across choices, the same 30 injected runs and the 15 control runs were reanalysed at thresholds from 5% to 50%. The true-positive rate (TPR) was computed on the 30 injected runs and the false-positive rate (FPR) on the 15 control runs. Table 5.5 reports the result at a few representative thresholds, and Figure 5.3 shows the full sweep.

TABLE 5.5  
Detector Quality vs. Regression Threshold

| Threshold | TPR  | FPR  | Precision | F1   | Comment                                |
|-----------|------|------|-----------|------|----------------------------------------|
| 0.05      | 1.00 | 0.13 | 0.95      | 0.98 | High recall, some false alerts         |
| 0.10      | 1.00 | 0.07 | 0.98      | 0.99 | Best F1 in the swept range             |
| 0.15      | 0.95 | 0.07 | 0.98      | 0.96 | Default threshold                      |
| 0.20      | 0.93 | 0.00 | 1.00      | 0.96 | First threshold with zero false alerts |
| 0.25      | 0.81 | 0.00 | 1.00      | 0.89 | Misses borderline low-intensity cells  |
| 0.30      | 0.76 | 0.00 | 1.00      | 0.86 | Conservative, prefers precision        |

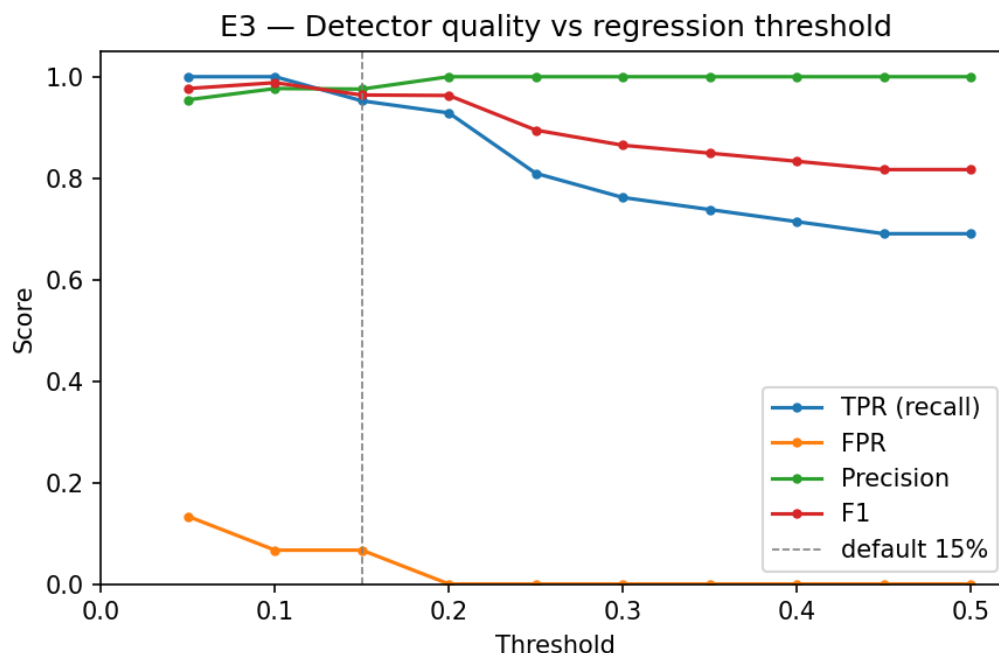


Fig. 5.3. Detector quality as a function of the relative-shift threshold. TPR and FPR are computed on the 30 injected runs and 15 control runs respectively. The vertical dashed line marks the default threshold of 15%.

The sweep shows two regions. Below 0.15 the detector finds every injected regression but also reports a small number of false alerts on the control runs: one or two false positives out of 15. At and above 0.20 the false-positive count drops to zero but recall falls as well, because the low-intensity CPU cells near the threshold are now missed.

The 15% default sits at the boundary between these two regions. It keeps recall above 95% while accepting one false positive in 15 control runs (6.7% FPR) on the data collected in this evaluation. A 0.20 threshold trades roughly two percentage points of recall for zero false alerts on this dataset. The choice between the two depends on the operating preference: a system tuned for catching as many regressions as possible should use 0.15 or lower, while a system tuned to minimise developer alert fatigue should use 0.20.

## 5.5 Discussion of Results

This section interprets the results in relation to the three research questions defined in Chapter 1.

The first research question asks how Android application performance can be monitored continuously with low runtime overhead. The results in Section 5.2 show that enabling the SDK added at most 5.2 percentage points of CPU usage, 5.4 MB of memory, and 75 ms of cold-start time across all tested devices. These costs are small enough that an end user is unlikely to perceive them, and are consistent with the background sampling, two-tier buffering, and batched transmission design described in Chapter 4.

The second research question asks how performance regression introduced by application updates can be detected automatically. The pipeline groups metric samples by version code, metric, screen, and device cohort, applies a maturity gate, and then flags a regression when the relative P95 shift between two consecutive releases exceeds the configured threshold. This approach requires no manual labelling and no per-metric threshold tuning, and it runs continuously against incoming production data.

The third research question asks how accurate the proposed system is in detecting performance regressions. The pipeline detected 27 of 30 injected runs (90%) across three regression types, three intensity levels, and three device cohorts. At the default 15% threshold, one false alert was produced in 15 control runs ( $\text{FPR} \approx 7\%$ ). Raising the threshold to 20% eliminates false alerts at the cost of roughly two percentage points of recall, as shown in Section 5.4.4. The three missed detections all fall in the low-intensity CPU configuration, where the P95 shift sits at or just below the threshold.

## 5.6 Limitations

The evaluation has several limitations.

First, the system was not deployed to real users. Overhead and detection quality were measured on a small fixed set of devices running one usage scenario, which does not capture the variety of real-world usage.

Second, emulators do not fully replicate the hardware behaviour of physical devices, so the emulator results for CPU and memory should be treated as indicative rather than conclusive.

Third, the device cohort is assigned using a simple rule based only on total RAM and CPU core count. Two devices with the same RAM and the same number of cores can still differ in real performance because of their chipset, GPU, or thermal behaviour, so this rule may place a device in the wrong cohort and weaken the cohort-based comparison.

## Chapter 6

# Conclusion

This thesis addressed performance monitoring and regression detection for Android applications after release. The problem it tackled is that existing tools either provide no automated alerting or rely on fixed thresholds that have to be set and maintained by hand for every metric. On a fragmented device base, such absolute thresholds produce frequent false or missed alerts, and a slowdown introduced by a new release can go unnoticed as long as it stays within the configured limit.

To address this, a complete system was designed and implemented. A mobile SDK collects performance metrics on real user devices with low overhead and uploads them to the backend in batches. A Ktor backend stores the data in ClickHouse and PostgreSQL and assigns a device cohort to each record. A detection pipeline then compares consecutive releases by the relative P95 shift, grouped by metric, screen, and cohort, and notifies developers when a new release degrades performance. The system was evaluated for runtime overhead and for the accuracy of its regression detection. The results, together with the answers to the research questions, are reported in Chapter 5.

The usefulness of the work lies in combining continuous post-release monitoring on real Android devices with automated, server-side detection of regressions between releases, a combination not offered by existing tools. Because detection is based on the relative change between two releases rather than on absolute limits, developers do not need to set or tune thresholds by hand. The comparison is also stratified by device cohort, so slower low-end hardware is not mistaken for a regression. The system is not intended to replace pre-release profiling tools. Instead, it complements them by covering the post-release period on real user devices.

## 6.1 Future Work

The most important next step is a real-world deployment on a live application with real users, which would test the system against naturally occurring regressions under varied hardware, usage patterns, and network conditions. The cohort classification could also be improved. Instead of relying on static hardware attributes such as RAM and CPU core count, cohorts could be derived from each device's observed performance, for example by grouping devices according to a lightweight on-device benchmark. Finally, the set of collected metrics could be extended with battery and network usage, which are also relevant to mobile user experience [17] but are not currently collected.

# Bibliography cited

- [1] Y. Tang, H. Wang, X. Zhan, *et al.*, “A systematical study on application performance management libraries for apps,” *IEEE Trans. Softw. Eng.*, vol. 48, no. 8, pp. 3044–3065, Aug. 2022, ISSN: 0098-5589. DOI: [10.1109/TSE.2021.3077654](https://doi.org/10.1109/TSE.2021.3077654).
- [2] Firebase. “Firebase performance monitoring.” Accessed: 30 Oct 2025. (2025), [Online]. Available: <https://firebase.google.com/docs/perf-mon>.
- [3] Z. Dong, Y. Zhao, T. Liu, *et al.*, “Same app, different behaviors: Uncovering device-specific behaviors in android apps,” in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '24, Sacramento, CA, USA: Association for Computing Machinery, 2024, pp. 2099–2109, ISBN: 9798400712487. DOI: [10.1145/3691620.3695272](https://doi.org/10.1145/3691620.3695272).
- [4] M. Kamran, J. Rashid, and M. Nisar, “Android fragmentation classification, causes, problems and solutions,” *International Journal of Computer Network and Information Security*, vol. 14, pp. 992–999, Sep. 2016.
- [5] H. Khalid, E. Shihab, M. Nagappan, and A. E. Hassan, “What do mobile app users complain about?” *IEEE Software*, vol. 32, no. 3, pp. 70–77, 2015. DOI: [10.1109/MS.2014.50](https://doi.org/10.1109/MS.2014.50).

- [6] X. Wei, L. Gomez, I. Neamtiu, and M. Faloutsos, “Profiledroid: Multi-layer profiling of android applications,” Aug. 2012. DOI: [10.1145/2348543.2348563](https://doi.org/10.1145/2348543.2348563).
- [7] S. Jeong, K. Lee, J.-Y. Hwang, S. Lee, and Y. Won, “Androstep: Android storage performance analysis tool,” in *Software Engineering*, 2013.
- [8] F. Bouaffar, O. L. Goaer, and A. Nouredine, “Powdroid: Energy profiling of android applications,” in *2021 36th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW)*, 2021, pp. 251–254. DOI: [10.1109/ASEW52652.2021.00055](https://doi.org/10.1109/ASEW52652.2021.00055).
- [9] W. Liu, F. Lin, L. Guo, T.-H. Chen, and A. Hassan, “Guiwatcher: Automatically detecting gui lags by analyzing mobile application screencasts,” Feb. 2025. DOI: [10.48550/arXiv.2502.04202](https://doi.org/10.48550/arXiv.2502.04202).
- [10] L. Zhang, M. Gordon, R. Dick, Z. Mao, P. Dinda, and L. Yang, “Adel: An automatic detector of energy leaks for smartphone applications,” Oct. 2012, pp. 363–372. DOI: [10.1145/2380445.2380503](https://doi.org/10.1145/2380445.2380503).
- [11] L. R. Sivalingam, J. Padhye, S. Agarwal, R. Mahajan, I. Obermiller, and S. Shayandeh, “Appinsight: Mobile app performance monitoring in the wild,” Nov. 2012.
- [12] Y. Luo, K. Rodrigues, C. Li, *et al.*, “Hubble: Performance debugging with In-Production, Just-In-Time method tracing on android,” in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, Carlsbad, CA: USENIX Association, Jul. 2022, pp. 787–803, ISBN: 978-1-939133-28-1.

- [13] D. K. Hong, A. Nikraves, Z. M. Mao, M. Ketkar, and M. Kishinevsky, “Perfprobe: A systematic, cross-layer performance diagnosis framework for mobile platforms,” in *2019 IEEE/ACM 6th International Conference on Mobile Software Engineering and Systems (MOBILESoft)*, 2019, pp. 50–61. DOI: [10.1109/MOBILESoft.2019.00018](https://doi.org/10.1109/MOBILESoft.2019.00018).
- [14] L. Zhang, D. Bild, R. Dick, Z. Mao, and P. Dinda, “Panappticon: Event-based tracing to measure mobile application and platform performance,” Sep. 2013, pp. 1–10. DOI: [10.1109/CODES-ISSS.2013.6659020](https://doi.org/10.1109/CODES-ISSS.2013.6659020).
- [15] J. Fu, Y. Wang, Y. Zhou, and X. Wang, “How resource utilization influences ui responsiveness of android software,” *Information and Software Technology*, vol. 141, p. 106 728, Sep. 2021. DOI: [10.1016/j.infsof.2021.106728](https://doi.org/10.1016/j.infsof.2021.106728).
- [16] B. Li, Q. Zhao, S. Jiao, and X. Liu, “Droidperf: Profiling memory objects on android devices,” Jul. 2023, pp. 1–15. DOI: [10.1145/3570361.3592503](https://doi.org/10.1145/3570361.3592503).
- [17] R. Rua and J. Saraiva, “A large-scale empirical study on mobile performance: Energy, run-time and memory,” *Empirical Softw. Engg.*, vol. 29, no. 1, Dec. 2023, ISSN: 1382-3256. DOI: [10.1007/s10664-023-10391-y](https://doi.org/10.1007/s10664-023-10391-y).
- [18] Android Developers. “Slow rendering.” Accessed: 19 April 2026. (2026), [Online]. Available: <https://developer.android.com/topic/performance/vitals/render>.
- [19] Google Android. “App startup time.” Accessed: 30 Oct 2025. (2025), [Online]. Available: <https://developer.android.com/topic/performance/vitals/launch-time>.
- [20] Google Android. “Anrs.” Accessed: 30 Oct 2025. (2025), [Online]. Available: <https://developer.android.com/topic/performance/vitals/anr>.

- [21] Google Play Console. “Monitor your app’s technical quality with android vitals.” Accessed: 30 Oct 2025. (2025), [Online]. Available: <https://play.google.com/console/about/vitals/>.
- [22] T. H. D. Nguyen, B. Adams, Z. M. Jiang, A. E. Hassan, M. Nasser, and P. Flora, “Automated detection of performance regressions using statistical process control techniques,” in *Proceedings of the 3rd ACM/SPEC International Conference on Performance Engineering (ICPE ’12)*, ACM, 2012, pp. 299–310.
- [23] A. Oliner, A. Iyer, I. Stoica, E. Lagerspetz, and S. Tarkoma, “Carat: Collaborative energy diagnosis for mobile devices,” Nov. 2013. DOI: [10.1145/2517351.2517354](https://doi.org/10.1145/2517351.2517354).
- [24] Datadog. “Anomaly monitor.” Accessed: 30 Oct 2025. (2025), [Online]. Available: <https://docs.datadoghq.com/monitors/types/anomaly/>.
- [25] Netflix Technology Blog. “Automated canary analysis at Netflix with Kayenta.” (2018), [Online]. Available: <https://netflixtechblog.com/automated-canary-analysis-at-netflix-with-kayenta-3260bc7acc69> (visited on 06/09/2026).
- [26] D. Y. Yoon, Y. Wang, M. Yu, *et al.*, “FBDetect: Catching tiny performance regressions at hyperscale through in-production monitoring,” in *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP ’24)*, ACM, 2024.
- [27] N. Balasubramanian, A. Balasubramanian, and A. Venkataramani, “Energy consumption in mobile phones: A measurement study and implications for network applications,” in *Proceedings of the 9th ACM SIG-*

- COMM Conference on Internet Measurement (IMC '09)*, New York, NY, USA: Association for Computing Machinery, 2009, pp. 280–293. DOI: [10.1145/1644893.1644927](https://doi.org/10.1145/1644893.1644927).
- [28] S. K. Jensen, T. B. Pedersen, and C. Thomsen, “Time series management systems: A survey,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 29, no. 11, pp. 2581–2600, 2017. DOI: [10.1109/TKDE.2017.2740932](https://doi.org/10.1109/TKDE.2017.2740932).
- [29] Android Developers. “Android’s kotlin-first approach.” Accessed: 19 April 2026. (2026), [Online]. Available: <https://developer.android.com/kotlin/first>.
- [30] R. Schulze, T. Schreiber, I. Yatsishin, R. Dahimene, and A. Milovidov, “ClickHouse - lightning fast analytics for everyone,” *Proceedings of the VLDB Endowment*, vol. 17, no. 12, pp. 3731–3744, 2024. DOI: [10.14778/3685800.3685802](https://doi.org/10.14778/3685800.3685802).
- [31] A. Blanck, *Cheddar: Hacker news client for android*, <https://github.com/adrianblancode/Cheddar>, Accessed: 2025-06-09, 2024.